BRAIN API

Table of Contents

1. Examples
   - Signing In
   - Biometrics Sign In
   - Simulating an Alpha
   - Waiting for an Alpha Simulation to End and Retrieving Results
   - Getting Record Sets of Alpha Simulations
2. API Documentation
   - /authentication
   - /simulations
   - /alphas
   - /alphas/<alpha_id>/recordsets
   - /alphas/<alpha_id>/recordsets/<record set name>
3. Additional Details
   - /users/<userid>/activities/diversity
4. Troubleshooting common API error messages

The BRAIN platform sports a rich API library you can use to automate requests to perform certain actions, such as testing alpha expressions with a range of values.

This document will detail a few common actions, such as how to sign in and send an alpha for simulation using the API, as well as describe the API behind these actions. We will use Python's Requests library to illustrate these examples, but you can communicate with the API endpoints using any programming language you prefer.

Examples

Signing In

To sign in to the platform, invoke the /authentication endpoint using a POST request with a basic authorization header. An example using the Python Requests is shown below.

```
import requests
import json
from os.path import expanduser

# Create a session to persistently store the headers
s = requests.Session()

# Save credentials from JSON file in home directory into session
with open(expanduser('~/.brain_credentials'), 'r') as f:
    s.auth = tuple(json.load(f))

# Send a POST request to the /authentication API
response = s.post('https://api.worldquantbrain.com/authentication')
```

You can create a JSON file with the name '.brain_credentials' in your home directory to store your credentials for use in the code above. An example of the JSON file contents can be as follows:

["<email>","<password>"]

If the query is successful, a JSON Web Token (JWT) is returned that you can use as a header for subsequent requests and is cached by the session.

Biometrics Sign In

If you have biometrics sign in enabled, the above request will return with a "status code 401" response, as well as a header location that you need to access through the browser to proceed with the biometric authentication.

```
WWW-Authenticate: persona
Location: /authentication/persona?inquiry=inq_ZQZkqAXPqQL7Vym9aPELnghV
```

You will have to access the abovementioned location through the browser. In this example, the URL will be https://api.worldquantbrain.com/authentication/persona?

response URL and the Location header as below:

```
from urllib.parse import urljoin

# Check status code for next action
if response.status_code == requests.status_codes.codes.unauthorized:
    if response.headers["WWW-Authenticate"] == "persona":
        # Outputs the URL to access through the browser to complete
biometrics authentication
        input("Complete biometrics authentication and press any key to
continue: " + urljoin(response.url, response.headers["Location"]))
        s.post(urljoin(response.url, response.headers["Location"]))
    else:
        print("incorrect email and password")
```

Simulating an Alpha

After finishing authentication, you can send alphas for simulation by sending a JSON object consisting of the settings of the simulation and the expression through a POST request to the /simulations API endpoint.

```
simulation_data = {
    'type': 'REGULAR',
    'settings': {
        'instrumentType': 'EQUITY',
        'region': 'USA',
        'universe': 'TOP3000',
        'delay': 1,
        'decay': 15,
        'neutralization': 'SUBINDUSTRY',
        'truncation': 0.08,
        'maxTrade': 'ON',
        'pasteurization': 'ON',
        'testPeriod': 'P1Y6M',
        'unitHandling': 'VERIFY',
        'nanHandling': 'OFF',
        'language': 'FASTEXPR',
        'visualization': False,
```

```
simulation_response = s.post("https://api.worldquantbrain.com/simulations",
json=simulation_data)
```

Waiting for an Alpha Simulation to End and Retrieving Results

You can check the progress of the simulation of the alpha by sending a GET request to the URL provided by the HTTP response returned when sending an alpha for simulation, under the header “Location.” If the alpha is still in the midst of simulation, a “Retry-After” header is returned and the script should wait for the specified amount of seconds before querying the URL again.

Once the simulation finishes, the response to the GET request will include a JSON object in the body where you can retrieve the alpha id in the “alpha” field, which you can then pass on to the /alphas API endpoint to retrieve the result.

```
from time import sleep

simulation_progress_url = simulation_response.headers['Location']
finished = False
while True:
    simulation_progress = s.get(simulation_progress_url)
    if simulation_progress.headers.get("Retry-After", 0) == 0:
        break
    print("Sleeping for " + simulation_progress.headers["Retry-After"] + "
seconds")
    sleep(float(simulation_progress.headers["Retry-After"]))
print("Alpha done simulating, getting alpha details")
alpha_id = simulation_progress.json()["alpha"]
alpha = s.get("https://api.worldquantbrain.com/alphas/" + alpha_id)
```

Getting Record Sets of Alpha Simulations

You can obtain the record set containing the information of your simulations, such as PnL and Sharpe ratio, for each trading day by sending a GET request to the /alphas/<alpha_id>/recordsets/<record set name>.

The example code below shows how to obtain PnL information over the simulation period

```
from time import sleep

finished = False
while True:
    pnl = s.get("https://api.worldquantbrain.com/alphas/" + alpha_id +
"/recordsets/pnl")
    if pnl.headers.get("Retry-After", 0) == 0:
        break
    print("Sleeping for " + pnl.headers["Retry-After"] + " seconds")
    sleep(float(pnl.headers["Retry-After"]))
print("PnL retrieved")
```

API Documentation

/authentication

GET

Retrieves the current state of the client's authentication.

Request:

```
GET /authentication
Cookie: jwt=<JWT>
```

Response:
If the client is not currently authenticated:

```
204 No Content
```

If the client is currently authenticated, then the user id and token expiry information is returned:

```
200 OK
{
    "user": {
        "id": "<string:user id>"
```

```
    }
    "permissions": [
        "PERMISSION",
        "..."
    ]
}
```

POST

Authenticates the client using basic authentication. The user may be required to solve a reCAPTCHA and then be locked out if they have had too many authentication attempts. The client is notified if a reCAPTCHA is required in the response to an unsuccessful POST.

If a user requires web authentication, a 401 message is returned with a response body containing the credential create or GET options.

Request:

```
POST /authentication
Authorization: Basic <Base64(user email:password)>
{
    "recaptcha": "<string:reCAPTCHA response if required>"
    "expiry": "<number:time until expiry in seconds, must be between 1 and
14400>"
}
```

Response:
If the authentication is successful, the authentication cookie is set and the user id and token expiry information is returned:

```
201 Created
Set-Cookie: t=<JWT>; httponly; Path=/; secure
{
    "user": {"id": "<string:user id>"},
    "token": {"expiry": <number:time until expiry in seconds>},
    "permissions": ["PERMISSION", "..."]
}
```

If the credentials are invalid:

If the credentials are invalid and a reCAPTCHA is required:

```
401 Unauthorized
{
    "detail": "<unauthorized code>",
    "recaptcha": [
        "This field is required."
    ]
}
```

DELETE

Deletes the authentication state by deleting the authentication cookie and invalidating the JWT.

Request:

```
DELETE /authentication
Cookie: t=<JWT>
```

Response:
If deleting authentication state is successful:

```
204 OK
Set-Cookie:t=; expires=Thu, 01-Jan-1970 00:00:00 GMT; Max-Age=0; Path=/
{
}
```

If the authentication status is invalid:

```
401 Unauthorized
{
    "detail": "INVALID_CREDENTIALS"
}
```

/simulations

values.

Request:

```
OPTIONS /simulations
```

Response:

```
200 Ok
{
    actions: {
        POST: {
            id: {
                type: "string",
                required: false,
                readOnly: true
            },
            ...
}
```

POST

Creates a new simulation.

Request:

```
POST /simulations
{
    "type": "<string: the simulation type: REGULAR or SUPER>",
    "settings": {
        "instrumentType": "<string: the simulation instrument type, see
OPTIONS for a list of available instrument type>",
        "region": "<string: the simulation region, see OPTIONS for a list of
available regions>",
        "universe": "<string: the simulation universe, see OPTIONS for a
list of available universes>",
        "delay": <number: 0 or 1 delay>,
        "decay": <number: the decay>,
        "neutralization": "<string: the simulation neutralization, see
OPTIONS for a list of available neutralizations>",
        "truncation": <number: the truncation>,
```

```
        "unitHandling": "<string: the simulation unit handling, see OPTIONS
for list of available unit handling>",
        "nanHandling": "<string: the simulation NaN handling, see OPTIONS
for list of available NaN handling>",
        "selectionHandling": "<string: the selection handling for the SUPER
simulations, see OPTIONS for a list of available selection handling>",
        "selectionLimit": "<string: the selection limit for the SUPER
simulations>",
        "language": "<string: the language, see OPTIONS for a list of
available languages>",
        "visualization": <boolean: the visualization>,
    }
    "regular": "<string: the code for the REGULAR simulation>"
    "combo": "<string: the combo for the SUPER simulation>"
    "selection": "<string: the selection for the SUPER simulation>"
}
```

Multiple simulations can be run by posting an array of length 2..10 of the above simulation objects. The user requires the MULTI_SIMULATION permission to allow multiple simulations. See /authentication to get the user's permissions. Also the settings for the simulation must be compatible, they must have the same simulation type, instrument type, region, delay and language.

```
[
    {"type":"REGULAR",...},
    {"type":"REGULAR",...},
    ...
]
```

Progress of multi-simulations is tracked by a parent simulation object. A child simulation is created for each of the multi-simulation objects. The list of child simulation ids are available when the parent simulation is complete.

Response:
If the simulation request was processed successfully, you will get the following response:

```
201 Created
Location: /simulations/<simulation id>
```

Updates to the simulation can be obtained from GET /simulations/<simulation id>. If a

```
400 Bad Request
{
    "type": [
        "\"X\" is not a valid choice."
    ],
    "settings": {
        "region": [
            "This field is required."
        ]
    }
}
```

GET

Retrieves the current state of simulation.

Request:

```
GET /simulations/<simulation id>
```

Response:
If the simulation is still in progress, the progress is returned:

```
200 OK
Retry-After: <seconds>
{
    "progress": <number: 0..1 progress of the asynchronous request if
available>
}
```

If the simulation is complete, the response body matches the POST response body with additional properties below.

If multiple simulations were submitted, the child simulation ids will be included. The details of the child simulations are available at GET /simulations/<child simulation id>

```
200 OK
{
    "id": "<string:simulation id>"
```

```
        "<string:simulation id>",
        ...
    ],
    "status": "<string: the simulation status: WAITING, SIMULATING, CANCELLED, COMPLETE, WARNING, ERROR, TIMEOUT, or FAIL>",
    "message": "<string:error message, if the status = ERROR and a message is available>"
    "location": { <object: code location if status = ERROR and the error is related to the code>
        "property": "<string: the code property with the error: 'regular', 'combo', or 'selection'>"
        "line": <number: the line number of the error in the code>,
        "start": <number: the start column of the error in the line>,
        "end": <number: the end column of the error in the line>,
    }
    "progress": <number: 0..1 simulation progress if the status = SIMULATING>
    "alpha": "<string: the id of the generated Alpha only available when status = COMPLETE or WARNING and simulation type = SIMULATE>"
    ...<the request object data>...
}
```

/alphas

This API endpoint manages alphas. Alphas are created by submitting simulations. See /simulations.

GET

Retrieves the alpha. Alpha ids can be obtained from the alpha of a simulation response object.

Request:

```
GET /alphas/<alpha id>
```

Response::

```
200 OK
```

```
        …
    }
    "regular": {
        …
    },
    "combo": {
        …
    },
    "selection": {
        …
    },
    …
}
```

/alphas/<alpha_id>/recordsets

GET

Request:

```
GET /alphas/<alpha id>/recordsets
```

Lists the record sets available for the Alpha.

Response:

```
200 OK
{
    "count": <number: the number of record sets>,
    "results": [
        {
            "name": "<string: the name of the record set>",
            "title": "<string: the human readable name of the record set>"
        },
        ...
    ]
}
```

Example including all record sets:

```
{
    "count": 19,
    "results": [
        {
        "name": "pnl",
        "title": "PnL"
        },
        {
        "name": "sharpe",
        "title": "Sharpe"
        },
        {
        "name": "turnover",
        "title": "Turnover"
        },
        {
        "name": "daily-pnl",
        "title": "Daily PnL"
        },
        {
        "name": "yearly-stats",
        "title": "Yearly Stats"
        },
        {
        "name": "coverage",
        "title": "Coverage of Universe"
        },
        {
        "name": "coverage-by-industry",
        "title": "Coverage by Industry"
        },
        {
        "name": "coverage-by-sector",
        "title": "Coverage by Sector"
        },
        {
        "name": "average-size-by-industry",
        "title": "Average Size by Industry"
        },
        {
```

```
        {
        "name": "average-size-by-capitalization",
        "title": "Average Size by Capitalization"
        },
        {
        "name": "pnl-by-industry",
        "title": "PnL by Industry"
        },
        {
        "name": "pnl-by-sector",
        "title": "PnL by Sector"
        },
        {
        "name": "pnl-by-capitalization",
        "title": "PnL by Capitalization"
        },
        {
        "name": "sharpe-by-industry",
        "title": "Sharpe by Industry"
        },
        {
        "name": "sharpe-by-sector",
        "title": "Sharpe by Sector"
        },
        {
        "name": "sharpe-by-capitalization",
        "title": "Sharpe by Capitalization"
        },
        {
        "name": "average-value-by-industry",
        "title": "Abs Average Value by Industry"
        },
        {
        "name": "average-value-by-sector",
        "title": "Abs Average Value by Sector"
        }
    ]
}
```

Request:

```
GET /alphas/<alpha id>/recordsets/<record set name>
```

Response:
Retrieves the given record set.

Depending on the record set there will be different properties:

```
pnl: ... date , pnl , sharpe , ...
sharpe: ... date , sharpe , ...
coverage: ... date , instruments , ...
coverage-by-industry: ... series , software , chemicals , ... the industries
...
coverage-by-sector: ... series , technology , financial , ... the sector ...
average-size-by-capitalization: ... 020 , 2040 , 4060 , 6080 , 80100 , ...
average-size-by-sector: ... technology , financial , ... the sectors ...
average-size-by-industry: ... software , chemicals , ... the industries ...
pnl-by-capitalization: ... date , pnlDivTwo , 020 , 2040 , 4060 , 6080 ,
80100 , ...
pnl-by-sector: ... date , pnlDivTwo , technology , basicMaterials , ... the
sectors ...
pnl-by-industry: ... date , pnlDivTwo , software , chemicals , ... the
industries ...
sharpe-by-capitalization: ... 020 , 2040 , 4060 , 6080 , 80100 , ...
sharpe-by-sector: ... technology , basicMaterials , ... the sectors ...
sharpe-by-industry: ... software , chemicals , ... the industries ...
average-value-by-sector: ... date , technology , basicMaterials , ... the
sectors ...
average-value-by-industry: ... date , software , chemicals , ... the
industries ...
turnover: ... date , turnover , ...
```

Additional Details

Record Sets

Some responses include tabular record sets. These can be used for tables and charting.

```
{
    "schema": {
        "name": "<string: the name of the record set>",
        "title": "<string: the human readable name of the record set>",
        "properties": [ <array: the record set properties>
            {
                "name": "<string: the name of the property>"
                "title": "<string: the human readable name of the property>"
                "type": "<string: data type of the property: string,
integer, decimal, amount, percent, permyriad, date, time, datetime, year>"
            },
        ...]
    "records": [ <array: the records>
        [
            <array: the record property values>
        ],
    ...]
}
```

The type can be used by the client to format numbers. Amounts are generally formatted in thousands "1.2K", millions "1.2M", billions "1.2B". Percent and permyriad (basis points) are formatted as "1.23%", "1.23‰" respectively.

/users/<userid>/activities/diversity

GET

Request:

/users/<userid>/activities/diversity

Provide alpha submission breakdown by Region, Delay, and Data Category.

Return recordset with parameters:

/users/<userid>/activities/diversity?grouping=region,delay,dataCategory

Alpha shall belong to a Dataset Category if it uses a data field in a dataset from the Dataset Category.

Response:

'alphas': [

{

'alphaCount': <integer of alpha count>,

'delay': <integer of delay>,

'region': <string of region>,

'dataCategory': {'name': <string name of data category>, 'id': <string id of data category> }

},

...

],

'count': <integer of all alpha count>

}

Troubleshooting common API error messages

Error	Likely cause & resolution
Invalid data field	A data field was wrongly used in your Alpha expression. Check if you are using vector data fields with vector operators, or if you have included a matrix data field in a vector operator.
Empty output from ace_lib function	Some of your inputs might be invalid. Check through your inputs to the function. If the invalid input is a dataset or data field, check if the dataset or data field is still on the platform through the Data Explorer.
Got invalid input at index 0, must be an event data	An invalid input was detected. Check if you are passing a constant into a vector operator.

Prev: ❗Understanding simulation limits

Next: How can you avoid duplicate simulations?